```
import io
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import streamlit as st

from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

st.set_page_config(page_title="Clustering Cacat Produk Manufaktur", layout="wide"

# ------------------------------------------------------------------------
# Judul & Deskripsi
# ------------------------------------------------------------------------
st.title("🏭 Analisis Clustering Cacat Produk Industri Manufaktur")
st.caption(
    "Aplikasi ini mengelompokkan data cacat produk manufaktur menggunakan "
    "algoritma **K-Means Clustering** untuk membantu tim quality control "
    "memahami pola kegagalan produksi."
)

CATEGORICAL_FEATURES = ["defect_type", "defect_location", "severity", "inspection
NUMERIC_FEATURES = ["repair_cost", "month"]

# ------------------------------------------------------------------------
# 1. Load Data
# ------------------------------------------------------------------------
st.header("1. Data Understanding")

with st.sidebar:
    st.header("⚙️ Pengaturan")
    uploaded = st.file_uploader("Upload dataset (.csv) - opsional", type=["csv"])
    st.caption(
        "Jika tidak upload file, aplikasi memakai dataset contoh "
        "`defects_data.csv` yang sudah disertakan."
    )

@st.cache_data
def load_default_data():
    return pd.read_csv("defects_data.csv")
```

```
if uploaded is not None:
    df_raw = pd.read_csv(uploaded)
    st.success("Dataset kustom berhasil dimuat.")
else:
    df_raw = load_default_data()
    st.info("Menggunakan dataset contoh bawaan (defects_data.csv).")

df = df_raw.copy()

# Ekstraksi fitur bulan dari tanggal (jika ada kolom tanggal)
date_col = next((c for c in df.columns if "date" in c.lower()), None)
if date_col:
    df[date_col] = pd.to_datetime(df[date_col], errors="coerce")
    df["month"] = df[date_col].dt.month
else:
    df["month"] = 1

col1, col2 = st.columns([2, 1])
with col1:
    st.subheader("Pratinjau Data")
    st.dataframe(df.head(10), use_container_width=True)
with col2:
    st.subheader("Ringkasan")
    st.metric("Jumlah baris", df.shape[0])
    st.metric("Jumlah kolom", df.shape[1])
    st.write("**Missing value:**")
    st.dataframe(df.isnull().sum().rename("jumlah_null"))

missing_cat = [c for c in CATEGORICAL_FEATURES if c not in df.columns]
if missing_cat:
    st.error(
        f"Dataset tidak memiliki kolom wajib: {missing_cat}. "
        f"Pastikan dataset memiliki kolom: {CATEGORICAL_FEATURES + ['repair_cost'
    )
    st.stop()

# ------------------------------------------------------------------------
# 2. EDA singkat
# ------------------------------------------------------------------------
st.header("2. Exploratory Data Analysis (EDA)")

eda_col1, eda_col2 = st.columns(2)
with eda_col1:
    fig, ax = plt.subplots(figsize=(5, 3.5))
    df["defect_type"].value_counts().plot(kind="bar", ax=ax, color="#4C72B0")
    ax.set_title("Distribusi Jenis Cacat (defect_type)")
```

```
    ax.set_ylabel("Jumlah")
    st.pyplot(fig)

with eda_col2:
    fig2, ax2 = plt.subplots(figsize=(5, 3.5))
    sns.histplot(df["repair_cost"], kde=True, ax=ax2, color="#DD8452")
    ax2.set_title("Distribusi Biaya Perbaikan (repair_cost)")
    st.pyplot(fig2)

# ------------------------------------------------------------------------
# 3. Preprocessing
# ------------------------------------------------------------------------
st.header("3. Preprocessing")
st.write(
    "- Fitur numerik (`repair_cost`, `month`) dinormalisasi dengan **StandardScal
    "- Fitur kategorik (`defect_type`, `defect_location`, `severity`, `inspection
    "diubah dengan **One-Hot Encoding**"
)

features = CATEGORICAL_FEATURES + NUMERIC_FEATURES
features = [f for f in features if f in df.columns]
X = df[features]

preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), [f for f in NUMERIC_FEATURES if f in df.columns]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), [f for f in CATEGORICAL_FEATU
])

# ------------------------------------------------------------------------
# 4. Tentukan jumlah cluster optimal (Elbow Method)
# ------------------------------------------------------------------------
st.header("4. Menentukan Jumlah Cluster Optimal (Elbow Method)")

max_k = st.slider("Maksimum K untuk elbow method", 3, 15, 10)

@st.cache_data(show_spinner=False)
def compute_wcss(X_data, max_k, _preprocessor):
    processed = _preprocessor.fit_transform(X_data)
    if hasattr(processed, "toarray"):
        processed = processed.toarray()
    wcss = []
    for k in range(1, max_k + 1):
        km = KMeans(n_clusters=k, init="k-means++", random_state=42, n_init=10)
        km.fit(processed)
        wcss.append(km.inertia_)
    return wcss, processed
```

```
wcss, processed_X = compute_wcss(X, max_k, preprocessor)

fig3, ax3 = plt.subplots(figsize=(7, 3.5))
ax3.plot(range(1, max_k + 1), wcss, marker="o", linestyle="--")
ax3.set_title("Elbow Method for Optimal K")
ax3.set_xlabel("Number of Clusters")
ax3.set_ylabel("WCSS")
st.pyplot(fig3)

optimal_k = st.slider("Pilih jumlah cluster (K) berdasarkan grafik elbow di atas"

# ------------------------------------------------------------------------------
# 5. Fit KMeans final & Visualisasi PCA
# ------------------------------------------------------------------------------
st.header("5. Hasil Clustering")

kmeans = KMeans(n_clusters=optimal_k, init="k-means++", random_state=42, n_init=1
df["Cluster"] = kmeans.fit_predict(processed_X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(processed_X)
df["PCA1"] = X_pca[:, 0]
df["PCA2"] = X_pca[:, 1]

fig4, ax4 = plt.subplots(figsize=(7, 5))
sns.scatterplot(
    x="PCA1", y="PCA2", hue="Cluster", data=df, palette="viridis", s=45, ax=ax4
)
ax4.set_title("Defect Clusters (PCA Visualization)")
st.pyplot(fig4)

st.caption(
    f"Variansi yang dijelaskan 2 komponen PCA: "
    f"{pca.explained_variance_ratio_.sum()*100:.1f}%"
)

# ------------------------------------------------------------------------------
# 6. Ringkasan & Interpretasi Karakteristik Cluster
# ------------------------------------------------------------------------------
st.header("6. Interpretasi Hasil & Insight Bisnis")

def safe_mode(s):
    m = s.mode()
    return m.iloc[0] if not m.empty else "-"

agg_dict = {"repair_cost": ["mean", "median", "count"]}
for c in CATEGORICAL_FEATURES:
```

```
    if c in df.columns:
        agg_dict[c] = safe_mode

cluster_summary = df.groupby("Cluster").agg(agg_dict).round(2)
st.dataframe(cluster_summary, use_container_width=True)

st.subheader("📌 Insight per Cluster")
overall_mean_cost = df["repair_cost"].mean()

for cl in sorted(df["Cluster"].unique()):
    sub = df[df["Cluster"] == cl]
    mean_cost = sub["repair_cost"].mean()
    dominant_type = safe_mode(sub["defect_type"]) if "defect_type" in df.columns
    dominant_loc = safe_mode(sub["defect_location"]) if "defect_location" in df.c
    dominant_sev = safe_mode(sub["severity"]) if "severity" in df.columns else "-
    dominant_insp = safe_mode(sub["inspection_method"]) if "inspection_method" in
    cost_level = "tinggi" if mean_cost > overall_mean_cost * 1.15 else (
        "rendah" if mean_cost < overall_mean_cost * 0.85 else "menengah"
    )

    st.markdown(
        f"**Cluster {cl}** — {len(sub)} kasus, rata-rata biaya perbaikan "
        f"Rp {mean_cost:,.0f} (tergolong **{cost_level}**)\n"
        f"- Jenis cacat dominan: `{dominant_type}`\n"
        f"- Lokasi cacat dominan: `{dominant_loc}`\n"
        f"- Tingkat keparahan dominan: `{dominant_sev}`\n"
        f"- Metode inspeksi dominan: `{dominant_insp}`\n"
    )

st.subheader("💡 Rekomendasi Bisnis")
st.markdown(
    """
- **Cluster dengan biaya perbaikan tinggi & severity kritis** perlu diprioritaska
  root-cause analysis lebih lanjut, karena berdampak besar pada biaya produksi.
- **Cluster dengan inspeksi manual dominan** dapat dievaluasi untuk migrasi ke **
  testing**, guna mengurangi variabilitas deteksi cacat dan mempercepat proses QC
- **Cluster cacat structural pada lokasi surface** mengindikasikan potensi masala
  tahap produksi/perakitan tertentu — perlu audit proses di titik tersebut.
- Hasil clustering ini dapat dipakai sebagai dasar **maintenance preventif** dan
  anggaran perbaikan yang lebih efisien.
"""
)

# ---------------------------------------------------------------------------
# 7. Download hasil
# ---------------------------------------------------------------------------
st.header("7. Unduh Hasil")
```

```
csv_buffer = io.StringIO()
df.to_csv(csv_buffer, index=False)
st.download_button(
    "⬇️ Download data dengan label cluster (.csv)",
    data=csv_buffer.getvalue(),
    file_name="hasil_clustering_cacat_produk.csv",
    mime="text/csv",
)

st.divider()
st.caption("Dibuat untuk UAS Project Kecerdasan Buatan — Analisis Clustering Caca
```